Advanced policy search

Tutorial 12

Alejandro Garcia

11.1 Soft actor critic

1. Soft actor critic is closely related to entropy regularized policies. In these methods, the policy maximizes a quantity related to the expected (soft) Q-function plus an entropy term. Write down the maximization problem for the action probabilities for the special case of a state with two actions, given the corresponding Q-values. Show that the solution is a soft-max policy. Use $\alpha = 1$.

1. We have two actions, let's call the probability for action 1 'p', so the probability for action 2 is $1-p$. Now, we want to find:

$$\max_p pQ(s,a_1) + (1-p)Q(s,a_2) - p\log p - (1-p)\log(1-p). \tag{1}$$

We can do so by setting the gradient to zero:

$$\nabla_p pQ(s,a_1) + (1-p)Q(s,a_2) - p\log p - (1-p)\log(1-p) = 0 \tag{2}$$

$$Q(s,a_1) - Q(s,a_2) - \frac{p}{p} - \log p + \frac{1-p}{1-p} + \log(1-p) = 0, \tag{3}$$

thus

$$\log(1-p) - \log p = Q(s,a_2) - Q(s,a_1) \tag{4}$$

$$\log\frac{1-p}{p} = Q(s,a_2) - Q(s,a_1) \tag{5}$$

$$\frac{1}{p} - 1 = \exp(Q(s,a_2) - Q(s,a_1)) \tag{6}$$

$$p = \frac{1}{\exp(Q(s,a_2) - Q(s,a_1)) + 1} = \frac{\exp Q(s,a_1)}{\exp Q(s,a_1) + \exp Q(s,a_2)}. \tag{7}$$

The probability of action 2 is

$$1-p = \frac{\exp Q(s,a_2)}{\exp Q(s,a_1) + \exp Q(s,a_2)}.$$

We recognize this distribution over the actions as a softmax of the Q values.

2. In the diagram below, calculate the optimizing SAC policy for all states. Hint: first calculate the optimizing policy at states 2 and 3 with the soft Q function at those states, and then proceed with state 1.

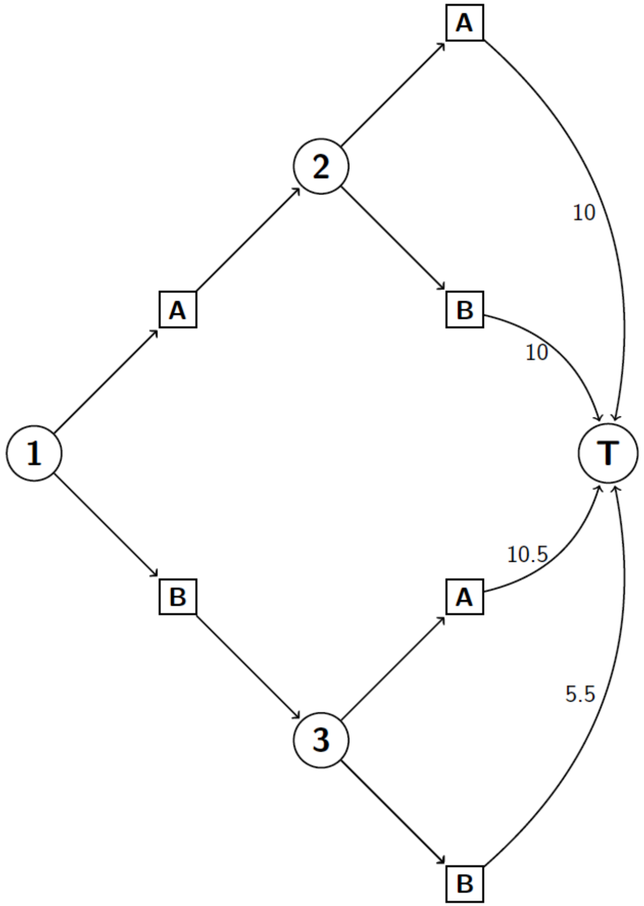

Figure 1: Markov Decision Process showing numbered states and actions A and B. Rewards are 0 except where indicated. T indicates the terminal state.

2. • For state 3:

The Q-values of the actions are clearly 10.5 and 5.5. Inserting these in the equation of part (a) yields: $p = \exp Q(s, a_1)/(\exp Q(s, a_1) + \exp Q(s, a_2)) \approx 36315/(36315 + 244) \approx 0.99$.

We can use these values to calculate the soft Q value of action B in state 1: We will denote soft Q values as $\tilde{Q}$ to distinguish them from 'regular' (non-soft) Q values. The soft Q value $\tilde{Q}(1, B)$ (objective value, eq. (1) is equal to:

$$pQ(s, a_1) + (1 - p)Q(s, a_2) - p \log p - (1 - p) \log(1 - p) \tag{8}$$

$$\approx 0.99 \cdot 10.5 + 0.01 \cdot 5.5 - 0.99 \log 0.99 - (1 - 0.99) \log(1 - 0.99) \tag{9}$$

$$\approx 10.5 \tag{10}$$

• For state 2:

For the policy, we can recognize by symmetry the probabilities of action A and action B are both 0.5 (we could also follow the same procedure as for state 3).

For the soft Q value $\tilde{Q}(1, A)$, we can also follow the same procedure as above and find it is approximately 10.69.

• For state 1:

Again we follow the same procedure, using the soft Q values calculated previously ($\tilde{Q}(1, A) \approx 10.69$, $\tilde{Q}(1, B) \approx 10.5$). We find as probability for action A: $p = 0.55$.

3. Entropy-regularized policies work slightly differently. Here, the policy maximizes the expected (non-soft) Q-function plus the policy entropy term. Re-calculate the optimizing policy for state 1 (the optimizing policy will be the same in states 2 and 3, but you will need to calculate the non-soft Q^π values in state 1).

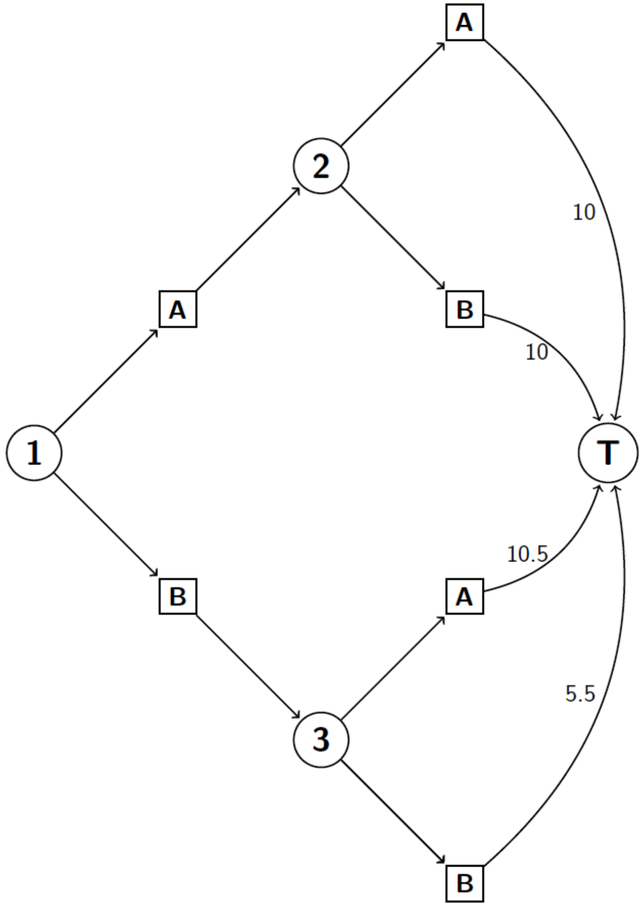

Figure 1: Markov Decision Process showing numbered states and actions A and B. Rewards are 0 except where indicated. T indicates the terminal state.

3. We now have to calculate the non-soft Q^π values $Q(1, A)$ and $Q(1, B)$. This is just the expected value of the reward under the policy in states 2, 3:

$$Q(1, A) = 0.5 \cdot 10 + 0.5 \cdot 10 = 10$$

$$Q(1, B) \approx 0.99 \cdot 10.5 + 0.01 \cdot 5.5 = 10.45$$

Plugging these in the softmax functions gives the probability of choosing 'A' in state 1 to be $\approx \exp(10)/(\exp(10) + \exp(10.45)) \approx 0.38$.

4. What is your conclusion about the difference between entropy-regularized policies and the policies found by soft actor critic?

4. Soft actor critic finds a policy that maximizes the entropy-regularized expected *soft* Q-values, whereas entropy-regularized RL maximizes entropy-regularized expected *regular (non-soft)* Q-values. In other words: soft actor critic doesn't only take the policy entropy of the current state into account, but also tries to reach future states where the policy entropy is high. This can lead to different policies, e.g., in the example the SAC policy would tend to prefer action A in state 1, whereas entropy-regularized policy gradients would tend to prefer action B in that state.

5. In this question, we have calculated the policy 'per state', whereas normally in the deep learning setting we cannot do that. Why not?

5. In the deep learning setting, we have function approximation, so weights might be shared between different actions / states, and the best parameter values for one state might not be the same as for another state, and some kind of intermediate value is reached through e.g. gradient descent. Here, we were assuming a tabular model.

11.2 Reward conditioned RL and deep learning

We will take a look at reward conditioned RL and trajectory stitching in a very simple model. Consider we have two states B and C. In state B, negative actions are rewarded and in state C, positive actions are rewarded. However, in the following set of trajectories, we do not have any examples where both actions are positively rewarded:

$$\{ (B, -1, 1, C, -1, -1, T), \tag{11}$$
$$(B, 1, -1, C, 1, 1, T), \tag{12}$$
$$(B, 1, -1, C, -1, -1, T)\}, \tag{13}$$

where each trajectory repeats state, (numeric) action, reward until reaching terminal state T.

1. Pre-process the data by calculating the Monte-Carlo return G for each (state,action) pair.

1. The pre-processed transitions are (note that we take each of the 6 transitions from the original data: 3 trajectories of 2 transitions each. This might lead to duplicate entries for the same (s,a) pair):

$$\{ (B, -1, 0) \tag{14}$$
$$(C, -1, -1), \tag{15}$$
$$(B, 1, 0), \tag{16}$$
$$(C, 1, 1), \tag{17}$$
$$(B, 1, -2), \tag{18}$$
$$(C, -1, -1)\}. \tag{19}$$

2. We will fit a model of the form

$$a = \begin{cases} c_B G & \text{if state is B} \\ c_C G & \text{if state is C} \end{cases}.$$

In contrast to the decision transformer and the decision diffuser in the lecture, here we predict the action only from the state and the return, and not from a longer history of states and actions. Write the model above as a linear regression model using binary features that indicate whether the state is equal to B and whether it is equal to C. For each transition, write down the ('input') features and the target values. Find the optimal coefficients c_1 and c_2 for the dataset.

The linear regression here is doable by hand - doing so is a useful practice for the exam.

2. We can write the linear regression model as:

$$a = c_B G \delta_B(S) + c_C G \delta_C(S),$$

with $\delta_B(S)$ and $\delta_C(S)$ indicator functions that are 1 if the state S is equal to B or C, respectively. Input features and targets (=action) for each row of the pre-processed transitions are:

$$\Phi = \begin{bmatrix} 0 & 0 \\ 0 & -1 \\ 0 & 0 \\ 0 & 1 \\ -2 & 0 \\ 0 & -1 \end{bmatrix}, \quad y = \begin{bmatrix} -1 \\ -1 \\ 1 \\ 1 \\ 1 \\ -1 \end{bmatrix}.$$

The least square solution is $c_B = -0.5, c_C = 1$. Thus, conditioning on the state being equal to B and a return of 1, we would predict $a = -0.5$. And conditioning on the state being equal to C and a return of 1, we would predict $a = 1$.

3. Explain how the finding relates to trajectory stitching.

3. Although we have never seen a trajectory with a negative action in state B and a positive action in state C, this is correctly predicted as rewarded sequence by the model.

4. The decision transformer takes as input a longer history of states and actions and predicts the next action. The decision diffuser predicts a whole future sequence of states. Why is this a good match for these architectures?

4. Transformers are good at predicting the next token in a sequence, exactly as they are being used in the diffusion transformer. Diffusion models are good at predicting high-dimensional variables - such as a whole sequence of states in case of the decision diffuser. Thus, these models are a good match to the problems as posed.